

GVR - Exercício MIB

GVR - Exercício MIB

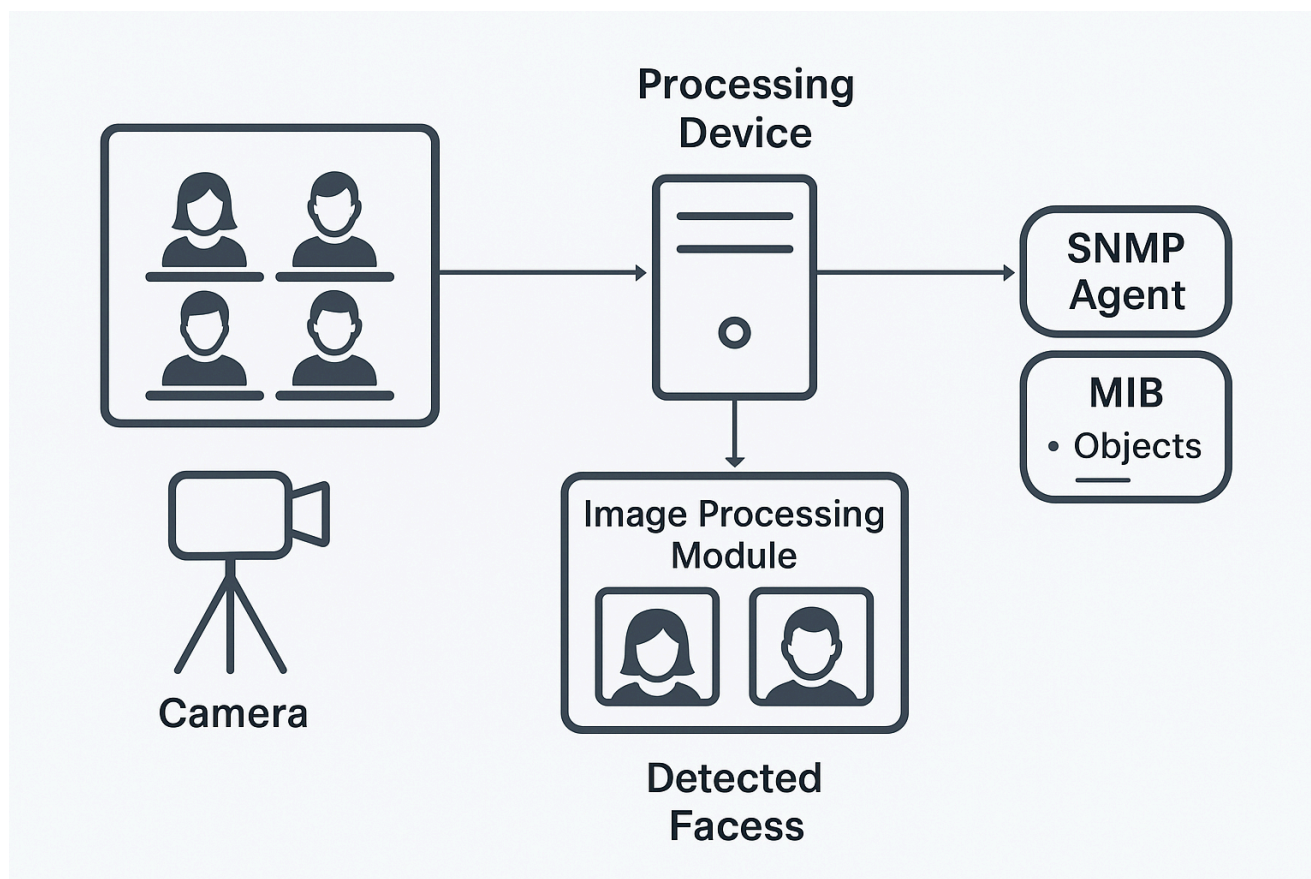
Problema

- ♦ registrar presenças dos alunos numa sala de aula via SNMP :-)

Pressupostos

- ♦ existe uma câmara de filmar que captura imagem da sala
- ♦ os alunos são identificados por módulo de identificação de rosto (vamos fingir que não há RGPD :-))
- ♦ os objetos são definidos numa MIB experimental
- ♦ agente SNMP simulado com snmsim
- ♦ estação de gestão com net-snmp (linha de comando)

Figura



Ferramentas a usar

- ♦ [SMI Tools](#) : para validar a MIB (validar e corrigir a syntaxe)
- ♦ [snmpsim](#) : simulador do agente snmp
- ♦ [net-snmp](#)  ou [PySNMP](#)  para clientes de linha (snmpget, snmpbulkget, etc)

Como instalar (Ubuntu ou Windows WSL):

Package	Como instalar	Descrição
SNMP Simulator	<code>pip install snmpsim</code>	snmpsim-command-responder, snmpsim-record-mibs
libsmi / smitools	<code>sudo apt install smitools</code>	smilint, smidump
Net-SNMP	<code>sudo apt install snmp snmp-mibs-downloader</code>	snmpget, snmpwalk, etc.
PySNMP	<code>pip install --upgrade pysnmp pysmi</code>	snmpget, snmpwalk, etc.

Ver o que está instalado no python:

```
pip show snmpsim pysnmp pysmi pysnmp-mibs snmpsim-data
```

Ver o que está obsoleto

```
pip list --outdated
```

Atualização para última versão (opcional)

```
pip install --upgrade snmpsim pysnmp pysmi pysnmp-mibs snmpsim-data
```

Como instalar (MacOS)

NOTA: usar *apenas uma* das ferramentas de gestão de pacotes [brew](#) ou [port](#), de acordo com a sua preferência

♦ ou com mac brew

```
brew install libsmi  
sudo port install libsmi
```

♦ ou com MacPorts

```
sudo port install libsmi
```

E depois instalar do mesmo modo os módulos python com `pip install...`

Teste rápido das ferramentas

```
smilint -h  
smidump -h
```

```
smidump -f tree IF-MIB
snmpget
snmpwalk
snmpsim-command-responder
```

Criação e teste da MIB

Sumário

Etapa 1: Planear a MIB (definir objetos, syntax, acesso) e a SMI (árvore de OIDs dos objetos definidos)

Etapa 2: Descarregar uma MIB exemplo NET-SNMP para facilitar a edição/criação da nova MIB

Etapa 3: Editar a MIB de acordo com o planeamento feito

Etapa 4: Validar a MIB com a ferramenta **smilint**

Etapa 5: Inventar dados para simular a MIB (à mão ou com gerador de dados)

Etapa 6 Executar o simulador - agente SNMP com os dados gerados

Etapa 7: Interrogar o agente (testes)

Etapa 1: Planear a MIB e a SMI

- ◆ Nome do módulo (em letras maiúsculas): CLASSROOM-MIB
- ◆ Prefixo a usar nos objetos (em letras minúsculas): classroom
- ◆ OID da raíz da MIB (top-level OID): abaixo de experimental(1.3.6.1.3), usando OID 2025

```
iso(1).org(3).dod(6).internet(1).experimental(3)
├─ classroomMIB(2025)
│   ├─ roomId(1)
│   └─ roomName(2)
│   ...
```

ACESSOS

MAX-ACCESS value	Meaning	Typical usage
not-accessible	Object exists for indexing or structure only; cannot be retrieved or set directly.	Table & entry nodes; index columns
read-only	Can only be read (GET, GETNEXT, GETBULK).	Counters, status, statistics
read-write	Can be read and modified (GET, SET).	Configurable parameters
read-create	Same as read-write, <i>plus</i> the ability to create new table rows.	Columns in tables that support dynamic creation (with RowStatus)
accessible-for-notify	Used only in notification definitions.	Trap/notification parameters

Objetos:

- ◆ Uma sala, tem um id (inteiro), um nome (string), capacidade (inteiro), estado (conjunto de valores), estado administrativo (conjunto de valores)
- ◆ Depois temos uma tabela com o numero do lugar, estudante que lá está (nº de estudante, nome, ano, curso)

Sala de aula:

- ◆ roomName (DisplayString), **read-write**
- ◆ roomCapacity (Integer32), **read-write**
- ◆ roomOperStatus (INTEGER { available(1), occupied(2), closed(3) }, **read-write** (runtime)
- ◆ roomAdminStatus (INTEGER { up(1), down(2) }, **read-write** (optional control)

Tabela de estudantes detetados pela camara e módulo de processamento de imagem:

- ◆ studentTable (table), studentEntry (row)
 - ◆ id (Unsigned32), not-accessible (index)
 - ◆ studentName (DisplayString), **read-create**
 - ◆ studentYear (INTEGER { first(1), second(2), third(3), fourth(4), fifth(5), other(99) }, **read-create**
 - ◆ studentCourse (DisplayString), **read-create**
 - ◆ status (RowStatus), **read-create** (para gerir esta linha da tabela: apagar, criar, alterar)

Etapa 2: Descarregar MIB exemplo

```
wget http://net-snmp.sourceforge.net/docs/mibs/NET-SNMP-EXAMPLES-MIB.txt
cp NET-SNMP-EXAMPLES-MIB.txt CLASSROOM-MIB.mib
```

Etapa 3: Editar MIB de acordo com o planeamento

```
CLASSROOM-MIB DEFINITIONS ::= BEGIN

--
-- MIB objects for classroom monitoring example implementations
--

IMPORTS
    MODULE-IDENTITY, OBJECT-TYPE, Integer32,
    NOTIFICATION-TYPE                FROM SNMPv2-SMI
    RowStatus, StorageType            FROM SNMPv2-TC
    InetAddressType, InetAddress     FROM INET-ADDRESS-MIB
;

-- MODULE IDENTITY: classromMIB
-- REVISION e LAST-UPDATED sao datas no formato AAAAMMDDHHMM
-- Tem de haver um REVISION igual a LAST-UPDATED

classroomMIB MODULE-IDENTITY
    LAST-UPDATED "202510160000Z"
    ORGANIZATION "www.net-snmp.org"
    CONTACT-INFO
```

```

    "postal:    Campus de Azurem
                4800-058 GUIMARAES
                Portugal
    email:     gvr@uminho.pt"

    "Aula de Gestao e Virtualizacao de Redes do MIETI"
DESCRIPTION
    "MIB objects for classroom monitoring example implementations"
REVISION     "202510160000Z"
DESCRIPTION
    "Classroom monitoring MIB: a single room (scalars) and a table of students."
    ::= { experimental 2025 }

-- Top-level branch for objects
classroomObjects OBJECT IDENTIFIER ::= { classroomMIB 1 }

-- ----- Textual Conventions -----
RoomOperStatus ::= INTEGER { available(1), occupied(2), closed(3) }
RoomAdminStatus ::= INTEGER { open(1), closed(2) }
StudentYear     ::= INTEGER { first(1), second(2), third(3), fourth(4),
fifth(5), other(99) }

-- =====
-- Room (single instance) – scalars
-- =====

roomName OBJECT-TYPE
    SYNTAX      DisplayString (SIZE (0..64))
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION  "Human-readable room name."
    ::= { classroomObjects 1 }

roomCapacity OBJECT-TYPE
    SYNTAX      Integer32 (0..10000)
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION  "Maximum occupants allowed in the room."
    ::= { classroomObjects 2 }

roomOperStatus OBJECT-TYPE
    SYNTAX      RoomOperStatus
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION  "Operational status of the room (runtime)."
    ::= { classroomObjects 3 }

roomAdminStatus OBJECT-TYPE
    SYNTAX      RoomAdminStatus
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION  "Administrative enable/disable of the room."
    ::= { classroomObjects 4 }

-- =====

```

```

-- Students (in the single room) - table
-- =====

studentTable OBJECT-TYPE
    SYNTAX          SEQUENCE OF StudentEntry
    MAX-ACCESS      not-accessible
    STATUS          current
    DESCRIPTION     "Students present/registered for the (single) room."
    ::= { classroomObjects 10 }

studentEntry OBJECT-TYPE
    SYNTAX          StudentEntry
    MAX-ACCESS      not-accessible
    STATUS          current
    DESCRIPTION     "One student row. Managed via RowStatus (create, modify,
delete).\"
    INDEX          { studentId }
    ::= { studentTable 1 }

StudentEntry ::=
    SEQUENCE {
        studentId      Unsigned32,
        studentName     DisplayString,
        studentYear     StudentYear,
        studentCourse    DisplayString,
        studentStatus    RowStatus
    }

studentId OBJECT-TYPE
    SYNTAX          Unsigned32
    MAX-ACCESS      not-accessible
    STATUS          current
    DESCRIPTION     "Row index (unique student identifier within this agent).\"
    ::= { studentEntry 1 }

studentName OBJECT-TYPE
    SYNTAX          DisplayString (SIZE (0..64))
    MAX-ACCESS      read-create
    STATUS          current
    DESCRIPTION     "Student full name.\"
    ::= { studentEntry 2 }

studentYear OBJECT-TYPE
    SYNTAX          StudentYear
    MAX-ACCESS      read-create
    STATUS          current
    DESCRIPTION     "Academic year.\"
    ::= { studentEntry 3 }

studentCourse OBJECT-TYPE
    SYNTAX          DisplayString (SIZE (0..64))
    MAX-ACCESS      read-create
    STATUS          current
    DESCRIPTION     "Course/program (e.g., MIETI).\"
    ::= { studentEntry 4 }

```

```

studentStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS   read-create
    STATUS       current
    DESCRIPTION
        "Row lifecycle: use createAndGo(4)/createAndWait(5) to create;
        set active(1)/notInService(2) to enable/disable; destroy(6) to delete."
    ::= { studentEntry 5 }

END

```

Etapa 4: Validar a MIB

```

smilint -s -m -l 6 -p /usr/share/snmp/mibs/ ./CLASSROOM-MIB.mib
snmptranslate -Td -OS CLASSROOM-MIB::studentTable

```

Nota: a opção -l 6 aumenta o nível de avisos que são enviados (warnings). O default é 3.

```

CLASSROOM-MIB DEFINITIONS ::= BEGIN

--
-- MIB objects for classroom monitoring example implementations
--

IMPORTS
    MODULE-IDENTITY, OBJECT-TYPE, Integer32, Unsigned32, experimental,
    NOTIFICATION-TYPE                                FROM SNMPv2-SMI
    RowStatus, StorageType, DisplayString             FROM SNMPv2-TC
    InetAddressType, InetAddress                     FROM INET-ADDRESS-MIB
;

-- MODULE IDENTITY: classromMIB
-- REVISION e LAST-UPDATED sao datas no formato AAAAMMDDHHMM
-- Tem de haver um REVISION igual a LAST-UPDATED

classroomMIB MODULE-IDENTITY
    LAST-UPDATED "202510160000Z"
    ORGANIZATION "www.net-snmp.org"
    CONTACT-INFO
        "postal:    Campus de Azurem
        4800-058 GUIMARAES
        Portugal
        email:      gvr@uminho.pt"
    DESCRIPTION
        "MIB objects for classroom monitoring example implementations"
    REVISION       "202510160000Z"
    DESCRIPTION
        "Classroom monitoring MIB: a single room (scalars) and a table of students."
    ::= { experimental 2025 }

```

```

-- Top-level branch for objects
classroomObjects OBJECT IDENTIFIER ::= { classroomMIB 1 }

-- Textual Conventions -----
RoomOperStatus ::= INTEGER { available(1), occupied(2), closed(3) }
RoomAdminStatus ::= INTEGER { open(1), closed(2) }
StudentYear      ::= INTEGER { first(1), second(2), third(3), fourth(4),
fifth(5), other(99) }

-- =====
-- Room (single instance) – scalars
-- =====

roomName OBJECT-TYPE
    SYNTAX      DisplayString (SIZE (0..64))
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION  "Human-readable room name."
    ::= { classroomObjects 1 }

roomCapacity OBJECT-TYPE
    SYNTAX      Integer32 (0..10000)
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION  "Maximum occupants allowed in the room."
    ::= { classroomObjects 2 }

roomOperStatus OBJECT-TYPE
    SYNTAX      RoomOperStatus
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION  "Operational status of the room (runtime)."
    ::= { classroomObjects 3 }

roomAdminStatus OBJECT-TYPE
    SYNTAX      RoomAdminStatus
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION  "Administrative enable/disable of the room."
    ::= { classroomObjects 4 }

-- =====
-- Students (in the single room) – table
-- =====

studentTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF StudentEntry
    MAX-ACCESS   not-accessible
    STATUS       current
    DESCRIPTION  "Students present/registered for the (single) room."
    ::= { classroomObjects 10 }

studentEntry OBJECT-TYPE
    SYNTAX      StudentEntry
    MAX-ACCESS   not-accessible

```



```

STATUS      current
DESCRIPTION  "One student row. Managed via RowStatus (create, modify,
delete)."
```

```

INDEX      { studentId }
::= { studentTable 1 }
```

```

StudentEntry ::=
SEQUENCE {
    studentId      Unsigned32,
    studentName    DisplayString,
    studentYear    StudentYear,
    studentCourse  DisplayString,
    studentStatus  RowStatus
}

studentId OBJECT-TYPE
SYNTAX      Unsigned32
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION  "Row index (unique student identifier within this agent)."
```

```

::= { studentEntry 1 }
```

```

studentName OBJECT-TYPE
SYNTAX      DisplayString
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION  "Student full name."
```

```

::= { studentEntry 2 }
```

```

studentYear OBJECT-TYPE
SYNTAX      StudentYear
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION  "Academic year."
```

```

::= { studentEntry 3 }
```

```

studentCourse OBJECT-TYPE
SYNTAX      DisplayString
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION  "Course/program (e.g., MIETI)."
```

```

::= { studentEntry 4 }
```

```

studentStatus OBJECT-TYPE
SYNTAX      RowStatus
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "Row lifecycle: use createAndGo(4)/createAndWait(5) to create;
    set active(1)/notInService(2) to enable/disable; destroy(6) to delete."
```

```

::= { studentEntry 5 }
```

```

END
```

Etapa 5: Inventar dados para simulação

Correr o mib compiler

```
python -m pysmi.tools.mibdump --help
```

ou

```
mibdump --help
```

Criar diretorias e compilar a MIB:

```
mkdir -p ~/snmp/mibs
mkdir -p ~/.pysnmp/mibs
mibdump --mib-source file://$HOME/snmp/mibs --mib-source
https://mibs.pysnmp.com/asn1/@mib@ --destination-directory ~/.pysnmp/mibs
CLASSROOM-MIB SNMPv2-MIB SNMPv2-SMI SNMPv2-TC
```

E depois gerar os dados:

```
snmpsim-record-mibs --mib-module CLASSROOM-MIB --output-file daclassroom-
auto.snmprec
```

Exemplo de dados:

```
# ===== Room scalars (use .0 for scalar instance) =====
1.3.6.1.3.2025.1.1.0|s|Room A
1.3.6.1.3.2025.1.2.0|i|30
1.3.6.1.3.2025.1.3.0|i|1      # roomOperStatus: available(1)
1.3.6.1.3.2025.1.4.0|i|1      # roomAdminStatus: up(1)

# ===== studentTable rows (INDEX { studentId }) =====
# studentId = 1
1.3.6.1.3.2025.1.10.1.2.1|s|Alice Silva
1.3.6.1.3.2025.1.10.1.3.1|i|2      # second(2)
1.3.6.1.3.2025.1.10.1.4.1|s|CS
1.3.6.1.3.2025.1.10.1.5.1|i|1      # RowStatus: active(1)

# studentId = 2
1.3.6.1.3.2025.1.10.1.2.2|s|Bruno Dias
1.3.6.1.3.2025.1.10.1.3.2|i|1      # first(1)
1.3.6.1.3.2025.1.10.1.4.2|s|CS
1.3.6.1.3.2025.1.10.1.5.2|i|1      # active(1)

# studentId = 7
1.3.6.1.3.2025.1.10.1.2.7|s|Carla Nunes
1.3.6.1.3.2025.1.10.1.3.7|i|3      # third(3)
1.3.6.1.3.2025.1.10.1.4.7|s|Math
1.3.6.1.3.2025.1.10.1.5.7|i|1      # active(1)
```

```
# studentId = 15
1.3.6.1.3.2025.1.10.1.2.15|s|Diego Costa
1.3.6.1.3.2025.1.10.1.3.15|i|5      # fifth(5)
1.3.6.1.3.2025.1.10.1.4.15|s|Physics
1.3.6.1.3.2025.1.10.1.5.15|i|1    # active(1)
```

Etapa 6: Executar o simulador

```
snmpsim-command-responder --data-dir ./data --agent-udp4-endpoint
127.0.0.1:2001
```

Etapa 7: Interrogar e testar o agente

```
snmpwalk -v2c -c public 127.0.0.1:2001 experimental.classroomMIB
snmpwalk -v2c -c public 127.0.0.1:2001 .1.3.6.1.3.2025
```

Perguntas

- ◆ Como podemos melhorar o modelo para em vez de apenas uma sala ter todas as salas do Campus? (redefinição da MIB)
- ◆ Como responder às seguintes perguntas (comandos SNMP):
 - ◆ "quantos lugares tem a sala"
 - ◆ "quantos alunos estão na sala"
 - ◆ "quais os números e nomes dos alunos nos 3 primeiros lugares sentados"
- ◆ Pense num algoritmo para marcar as presenças dos alunos na aula de GVR (por exemplo)

Referências

- ◆ [md_RFC 2579 Textual Conventions for SMIv2](#) 
- ◆ [md_libsmi - A Library to Access SMI MIB Information](#) 
- ◆ [md_Tools](#) 
- ◆ [md_Complete MIB Database - All SNMP MIBs and OIDs](#) 
- ◆ [md_net-snmpmibs](#) 
- ◆ [md_Net-SNMP](#) 
- ◆ [md_Net-SNMP](#) 
- ◆ [md_PySNMP 7 Homepage](#) 
- ◆ [md_SNMP Simulator Documentation](#) 



path: [As Minhas Smart Notes](#)

seeAlso: